❷ Within the handler, retrieve the Id of the selected property from the data-id attribute.

❸ Make the requests to the specified URL, as before.

❹ Call the text() method on the Response object to obtain the returned content as a string.

❺ Assign it to the modal body.

If you make these changes and run the application, you should not see any difference in behavior. The Fetch example is a little more verbose than the jQuery example, but it is not that complicated once you understand what's going on within the code.

Fetch works with *promises*, which are similar to .NET tasks. They represent the future result of an asynchronous operation. The Fetch HTTP call returns a response as a stream. Use the `text()` method to read the stream into a string, and then use the resulting string to update the modal body. If you are unfamiliar with it and would like to learn more about Fetch, I recommend the Mozilla Developer Network docs as a good starting point: http://mng.bz/vXO4.

Before we move on, it is essential to understand what goes on behind the scenes when these asynchronous calls are being made. You will use the browser developer tools to view the actual request being made and examine its response. The browser developer tools are an essential aid whenever you do client-side work because they reveal what is going on in the browser. You should refer to your browser's documentation for more information on how to access the tools and use them, but for Chrome and Edge users, all you have to do is press your F12 key when you have your browser open. Once the tools are exposed, click the Network tab that shows details of network traffic (requests), shown in figure 11.1. Then navigate to one of the city pages, and click a property name. You should see details of the request appear.